DWH MCP EXPLORER

РУКОВОДСТВО ПО ТЕХНИЧЕСКОЙ РЕАЛИЗАЦИИ

Stage 6.9.2 | DWH MCP Explorer | Русская версия

Базовая версия реализации 1.0 · 7 ноября 2026

Техническое описание проверенной архитектуры, интерфейса MCP и контрактов данных. Документ отражает реализованные функции, а не предполагаемые возможности.

1. Назначение и принципы проектирования

Python MCP-сервер предоставляет AI-клиенту фиксированный набор из 21 инструмента. Сервер является границей исполнения: Claude Desktop инициирует операции и интерпретирует структурированные ответы.

Контур PostgreSQL и SQL-репозитория предоставляет реляционные метаданные, SQL Lineage и сверку маппингов. Контур Azure Databricks предоставляет метаданные Unity Catalog, зарегистрированные события Lineage и точную проверку гранулярности.

Описанные операции читают метаданные или выполняют SQL только на чтение. Knowledge Base v1 формируется по запросу и не является постоянно хранимой индексированной базой.

Инженерные принципы

Граница	Принцип реализации
Тонкий адаптер	server.py регистрирует инструменты, а логика находится в модулях dwh_mcp/.
Тестируемость	Функции композиции получают провайдеры аргументами; внешние зависимости подменяются Mock.
Доказуемость	Сохраняются источники событий, счётчики и ограничения; события чтения отделяются от зависимостей.
Безопасность	Идентификаторы SQL валидируются; секреты хранятся вне кода; операции изменения данных не используются.

2. Архитектура выполнения и поток запросов

Обнаружение: Claude Desktop → локальное MCP-подключение → реестр инструментов server.py (21 имя).

PostgreSQL: MCP Tool → метаданные / SQL-репозиторий / Lineage / Reconciliation → структурированный результат.

Исследование Databricks: MCP Tool → Metadata Provider + Lineage Provider → Knowledge Object по запросу.

Проверка Grain: MCP Tool → валидация идентификаторов → SDK Statement Execution → SELECT в SQL Warehouse → отчёт.

```
Claude Desktop (MCP client)
      |
```

```
      v
server.py  --  21 MCP tools
      |
      +-- PostgreSQL / SQL repository
      |    metadata / SQL lineage / reconciliation
      |
      +-- Azure Databricks (SDK / OAuth)
           +-- Unity Catalog metadata
           +-- SQL Warehouse: table/column lineage
           +-- SQL Warehouse: grain SELECT
      |
      v
Structured response -> Claude technical report
```

3. Структура Python-кода

Путь / модуль	Назначение
server.py	Адаптер MCP, регистрация и вызов инструментов
dwh_mcp/database.py	Доступ к метаданным PostgreSQL
dwh_mcp/sql_repository.py	Поиск и чтение SQL-файлов
dwh_mcp/lineage.py	Извлечение SQL lineage
dwh_mcp/reconciliation.py	Проверка маппингов и ограничений
dwh_mcp/reporting.py	Формирование отчётов
dwh_mcp/composition.py	Композиция описания объектов DWH
dwh_mcp/models.py; errors.py	Модели и классификация ошибок
dwh_mcp/databricks_provider.py	Провайдер метаданных Unity Catalog
dwh_mcp/databricks_lineage_provider.py	SQL-запросы Lineage через Statement Execution
dwh_mcp/databricks_knowledge.py	Композиция объекта знаний по запросу
dwh_mcp/databricks_reconciliation.py	Проверка идентификаторов, сборка SQL и интерпретация Grain
dwh_mcp/databricks_grain_provider.py	Точная Grain-проверка через SQL Warehouse

4. Реестр MCP Tools и функциональные контракты

Регрессионные проверки MCP Client подтвердили ровно 21 зарегистрированное имя: 13 инструментов PostgreSQL/SQL/DWH и 8 инструментов Databricks. Имена параметров приведены только там, где они были проверены в ходе проекта. Для публичной публикации остальные сигнатуры нужно сверить с текущим Python-кодом.

4.1 PostgreSQL / SQL / DWH (13)

MCP Tool	Назначение
list_schemas	Список схем PostgreSQL
list_tables	Список таблиц в схеме
describe_table	Описание колонок и метаданных таблицы
get_primary_keys	Получение первичных ключей
get_foreign_keys	Получение внешних ключей
list_sql_files	Список SQL-файлов репозитория
read_sql_file	Чтение выбранного SQL-определения
get_table_knowledge	Сбор доступных знаний о таблице
get_table_lineage	Извлечение lineage из SQL-определений
reconcile_table_mapping	Сопоставление маппинга таблиц
reconcile_column_constraints	Сопоставление ограничений колонок
generate_reconciliation_report	Формирование отчёта о согласованности
investigate_dwh_object	Комплексное исследование объекта DWH

4.2 Azure Databricks (8)

MCP Tool	Назначение
databricks_list_catalogs	Список каталогов Unity Catalog
databricks_list_schemas	Список схем Unity Catalog
databricks_list_tables	Список таблиц Databricks
databricks_describe_table	Метаданные таблицы Databricks
databricks_get_table_lineage	События lineage на уровне таблиц
databricks_get_column_lineage	События lineage на уровне колонок
databricks_investigate_object	Сбор составного объекта знаний по запросу
databricks_validate_grain	Точная проверка ключа и гранулярности

5. Проверенные примеры вызова Databricks Tools

Ниже приведены сокращённые иллюстративные JSON-структуры, соответствующие проверенным входам и контрактам ответов. Это не полные выгрузки реальных результатов.

5.1 databricks_investigate_object

Вход: full_name (имя catalog.schema.table). Внутренние вызовы: describe_table, get_table_lineage и get_column_lineage. Результат объединяет метаданные, lineage на двух уровнях, источники, ограничения и статус.


```
{
 "object_name": "marketplace_lab.gold.fact_orders",
 "platform": "azure_databricks",
 "metadata": {"full_name": "...", "table_type": "MANAGED"},
 "table_lineage": {"status": "completed", "events": ["..."]},
 "column_lineage": {"status": "completed", "events": ["..."]},
 "execution_context": {},
 "evidence": {"metadata_source": "unity_catalog"},
 "limitations": ["Lineage coverage is bounded"],
 "status": "completed"
}
```


5.2 databricks_validate_grain

Вход: full_name и key_column. Провайдер формирует SQL только на чтение, выполняет его и вычисляет превышение числа строк: total_rows - distinct_keys - null_keys. Статус passed допустим только при нуле лишних строк и нуле NULL.

```
INPUT
{"full_name": "marketplace_lab.gold.fact_orders", "key_column": "order_id"}

OUTPUT (validated result)
{
 "table_name": "marketplace_lab.gold.fact_orders",
 "key_column": "order_id",
 "total_rows": 10000,
 "distinct_keys": 10000,
 "duplicate_rows": 0,
 "null_keys": 0,
 "status": "passed"
}
```

Read-only SQL / SQL только на чтение

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(DISTINCT `order_id`) AS distinct_keys,
  SUM(CASE WHEN `order_id` IS NULL THEN 1 ELSE 0 END) AS null_keys
FROM `marketplace_lab`.`gold`.`fact_orders`
```

6. Целостность данных, Lineage и ограничения

• Статистика таблиц Unity Catalog может быть приблизительной; нельзя делать окончательный вывод о дубликатах только по ней. Точный запрос к fact_orders показал 10 000 строк и 10 000 уникальных order_id, без NULL и лишних строк.

• Для Lineage используются system.access.table_lineage и system.access.column_lineage; текущие запросы смотрят назад на 45 дней и возвращают не более 50 событий за вызов. Даже truncated=false при 50 строках не гарантирует полноту графа.

• События с заполненными source и target подтверждают прямую зависимость. Событие без target может отражать чтение, а не downstream-зависимость. Column Lineage не содержит SQL-выражений преобразований.

• Поле execution_context в Knowledge Object v1 пусто. В событиях могут присутствовать entity_type/entity_id; разрешение имён заданий, путей notebook и SQL-текстов пока не реализовано.

• Проверка Grain поддерживает один простой столбец ключа и трёхчастное имя таблицы, но не составные ключи и не полный семантический Reconciliation между платформами.

7. Обработка ошибок, границы тестов и подтверждения

Область	Подтверждённый результат
Knowledge Composition	Проверены Mock-провайдеры, реальная интеграция Databricks и E2E через Claude Desktop.
Grain Validation	Проверены сборка SQL, Mock SDK, реальный SELECT, MCP Client и Claude Desktop.
Реестр / совместимость	21 инструмент обнаружен; PostgreSQL/DWH и Databricks зарегистрированы совместно.
Регрессия	Пройдены семь новых и пять старых тестов, проверка реестра и негативные сценарии.
Обработка ошибок	Искусственный RuntimeError возвращается MCP-клиенту с is_error=True.

О границах проверки: тесты с Mock не подтверждают доступность удалённых сервисов. Реальная проверка Databricks выполнена отдельно. Для PostgreSQL/SQL есть ранее созданные проверки, но документ не является полной сертификацией безопасности или нагрузочного тестирования.

8. Примечания для сопровождения

• Сохраняйте имена 21 инструмента стабильными, если изменение API не запланировано. В тестах реестра проверяйте наличие нужных имён, а не только точное количество.

• Локально используйте существующий Databricks CLI/OAuth профиль MARKETPLACE_LAB. Нельзя публиковать кэши токенов, секретные .env, приватную конфигурацию и учётные данные компьютера.

• SQL Warehouse может потреблять платные вычислительные ресурсы. В обычных автотестах используйте Mock-провайдеры; реальные интеграционные тесты запускайте отдельно.

• Перед публикацией проверьте текущий исходный код, зависимости и сигнатуры, подготовьте безопасные шаблоны конфигурации и выполните аудит секретов.

Основа: архитектурный документ Stage 6.9.1 EN/RU и результаты проверок Stage 5–6.